ENGINEERING DESIGN DOCUMENT

# Data Bus Component

High-Performance Message Transport Layer for Aegis OS

**Document ID:** EDD-AEGIS-DATABUS-001

**Version:** 2.0

**Author:** Meron Shibiru

**Project:** Aegis OS - Secure Infrastructure for Autonomous AI Agents

# Table of Contents


**1. Introduction**

1.1 Purpose

1.2 Scope

1.3 Definitions and Acronyms

**2. Architecture Overview**

2.1 The 6 JetStream Streams (Channels)

2.2 Communication Pseudocode

2.3 Position in System Architecture

2.4 Core Responsibilities

2.5 Design Principles

**3. Sequence Diagram**

3.1 Research Task — End-to-End Event Flow

**4. Interface Description**

4.1 Interface Overview

4.2 Interface Specifications

4.3 Standard Event Envelope (CloudEvents)

**5. Dependency Description**

5.1 Component Dependency Overview

5.2 Boot Sequencing from Dependencies

5.3 Failure Recovery

**6. Requirements Specification**

6.1 Functional Requirements

6.2 Non-Functional Requirements

6.3 Security Requirements

**7. Detailed Design**

7.1 Architecture Overview
7.2 Component Interactions
7.3 Message Flow and Sequences
7.4 Data Structures

**8. Communication Channels**

8.1 Channel Overview
8.2 Channel Pseudocode
8.3 Message Protocol

**9. OpenClaw Variant Analysis**

9.1 Comparative Analysis
9.2 Security Lessons (CVE-2026-25253)

**10. Proof of Concept**

10.1 PoC Components
10.2 PoC Pseudocode
10.3 PoC Test Cases

**11. High-Level User Task Scenario**

11.1 Task Definition
11.2 Event Flow

**12. Roadmap & Success Criteria**

12.1 Implementation Phases
12.2 Success Criteria by Phase

**13. Risks and Mitigation**

**14. Appendix**

# 1. Introduction

## 1.1 Purpose

This Engineering Design Document (EDD) defines the architecture, requirements, implementation approach, and proof-of-concept for the **Data Bus** component of Aegis OS. The Data Bus is the central nervous system of the entire platform, the asynchronous messaging backbone through which every other component communicates.

Unlike conventional message buses, the Aegis OS Data Bus is designed for the unique demands of an AI-agent-centric operating system, supporting complex message patterns including publish-subscribe, request-reply, streaming, and event sourcing, while maintaining strict security isolation between untrusted agent workloads.

## 1.2 Scope

This document covers:

- The 6 JetStream streams (channels) and how components communicate across them
- Full requirements specification (functional, non-functional, security)
- Sequence diagrams showing end-to-end event flow
- Interface descriptions (publish, subscribe, request-reply, metrics)
- Dependency diagram showing Data Bus as zero-dependency root
- Communication channel specifications with detailed pseudocode
- NATS JetStream variant analysis vs alternatives
- Proof-of-concept pseudocode and implementation plan
- High-level user task scenario exercising all components
- Implementation roadmap with success criteria

## 1.3 Definitions and Acronyms

| Term | Definition |
|---|---|
| Ack | Acknowledgment from consumer confirming successful message processing |
| CloudEvents | CNCF event envelope specification: specversion, id, source, type, time, data |
| Consumer | Stateful JetStream entity representing a named subscription into a Stream |
| Data Bus | Central message transport infrastructure enabling component communication |
| DLQ | Dead-Letter Queue — messages exceeding MaxDeliver are routed here |
| Durable | Consumer whose state persists across reconnections |
| JetStream | NATS persistence/streaming layer — adds durable Streams and Consumers |
| NKey | Ed25519 keypair used for NATS component authentication |
| Outbox | libSQL table storing events atomically with business data |
| Queue Group | Named subscriber group where each message is delivered to exactly one member |
| Replay | Replaying historical stream messages from a sequence offset or timestamp |
| Stream | Ordered, persistent log of messages on matching NATS subjects |
| Subject | Hierarchical routing key (e.g., aegis.tasks.routed) — supports * and > wildcards |

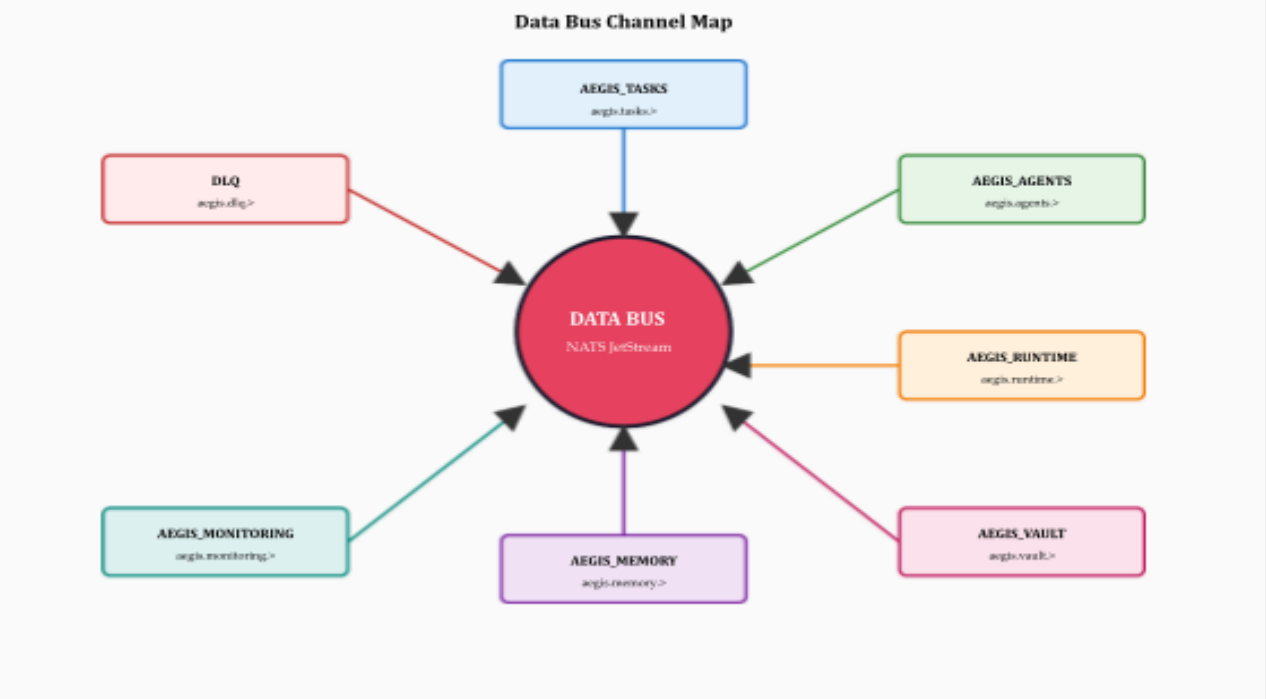
# 2. Architecture Overview

## 2.1 The 6 JetStream Streams (Channels)

The Data Bus is organised into six durable JetStream streams, each scoping a logical domain of the Aegis OS. Every stream carries at-least-once delivery guarantees, requires explicit consumer acknowledgment, and retains messages for 7 days / 10 GB.

| Stream (Channel) | Subject Pattern | Key Publishers | Key Subscribers |
|---|---|---|---|
| **AEGIS_TASKS** | aegis.tasks.> | Task Router, Task Planner | Task Planner, Agent Manager |
| **AEGIS_AGENTS** | aegis.agents.> | Agent Manager | Self-Healing, Memory Mgr, UI Layer |
| **AEGIS_RUNTIME** | aegis.runtime.> | Runtime Abstraction, Res. Allocator | Agent Manager, Monitoring |
| **AEGIS_VAULT** | aegis.vault.> | Permission Manager | Runtime Abstraction, Ext. Connectors |
| **AEGIS_MEMORY** | aegis.memory.> | Memory & Context Manager | Self-Healing, Knowledge Sharing |
| **AEGIS_MONITORING** | aegis.monitoring.> | Monitoring & Observability | All components (alert reactions) |

**Figure 2-1: Data Bus Channel Map — 6 JetStream Streams**



## 2.2 Communication Pseudocode

The following pseudocode illustrates how components communicate through the Data Bus at a high level. There are no direct calls between components — everything is mediated by the bus

**High-Level Communication Patterns**

```
// ── PUBLISHING (any component → Data Bus) ──────────────────────
component.publish(subject, envelope)
// envelope = { id, source, type, time, correlationId, data }
// Data Bus persists to JetStream stream, routes to all matching subscribers

// ── SUBSCRIBING (any component ← Data Bus) ──────────────────────
component.subscribe("aegis.tasks.routed", handler)
// handler(msg):
//     process(msg.data)
//     msg.ack()// explicit ack — removes from pending
//     msg.nak()          // negative ack — triggers redelivery after AckWait

// ── REQUEST-REPLY (synchronous over pub/sub) ────────────────────
response = component.request("aegis.config.get", { key: "max_agents" }, timeout=5s)
// Config Manager subscribes to aegis.config.get
// replies to auto-generated inbox subject
// caller blocks until reply received or timeout

// ── QUEUE GROUP (competing consumers / horizontal scale) ────
instance_A.queue_subscribe("aegis.tasks.routed", group="agent-managers", handler)
instance_B.queue_subscribe("aegis.tasks.routed", group="agent-managers", handler)
// Each TaskRouted message delivered to EXACTLY ONE instance
// Load distributed across all healthy group members

// ── WILDCARD SUBSCRIPTION (monitoring / self-healing) ──────────
monitoring.subscribe("aegis.>", record_all_events)
// Receives EVERY event from ALL 6 streams — read-only observer self_healing.subscribe("aegis.agents.failed",
trigger_recovery)
// Only receives specific failure events

// ── REPLAY (state reconstruction after node recovery) ──────────
consumer = data_bus.create_consumer(stream="AEGIS_AGENTS", deliver_from=10_minutes_ago)
// Replays historical messages — used by Self-Healing on restart

// ── OUTBOX PATTERN (zero-loss guarantee) ────────────────────────
```

```
BEGIN TRANSACTION
libsql.insert(outbox_table, subject, payload, status='pending')
libsql.commit()
END TRANSACTION
// Outbox Relay (background process):
ack = data_bus.publish(subject, payload)
libsql.update(outbox_table, status='sent', seq=ack.seq)
// If NATS is down, relay retries until publish succeeds
```

The Outbox Pattern guarantees that every state change in Aegis OS results in a corresponding Data Bus event, even across crashes, providing exactly-once *intent* with at-least-once delivery.

It **atomically couples state changes and event emission**.

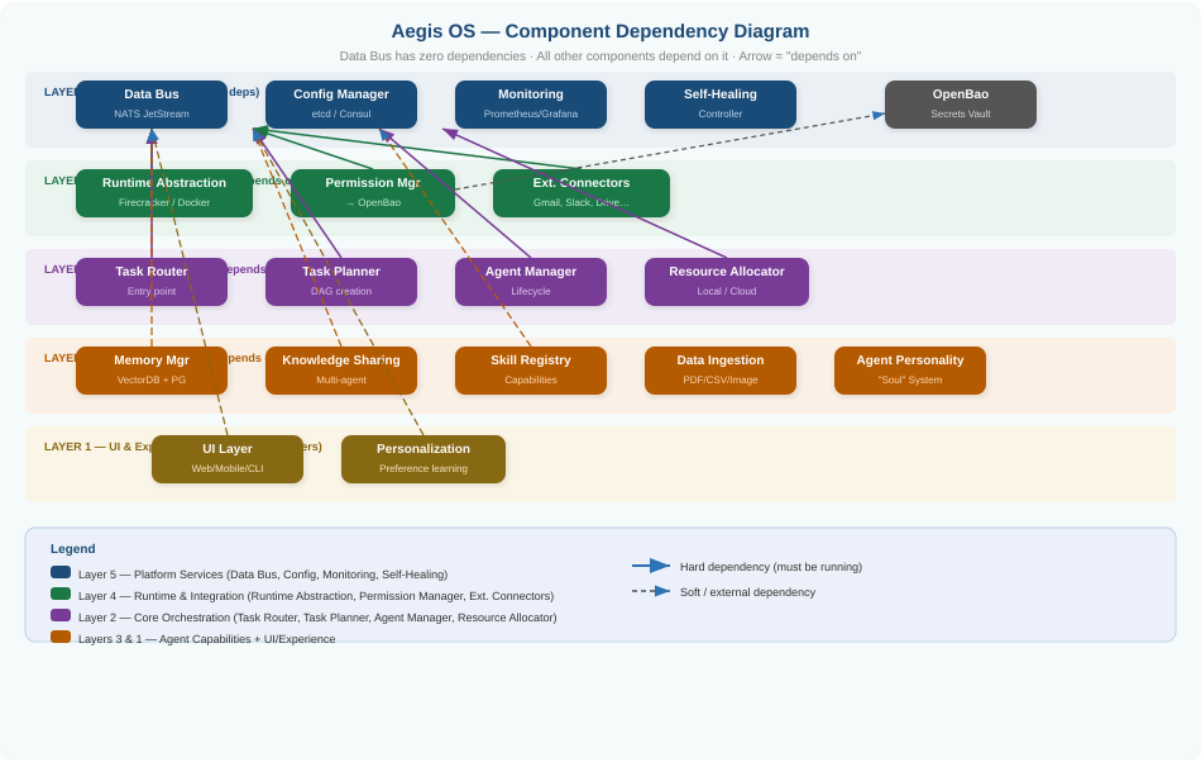
Instead of publishing directly to the Data Bus, a component:

1. Writes its state change **and** the event to a local durable store
2. A background relay publishes the event *after* the write commits
3. The event is only marked "sent" once acknowledged by the Data Bus

No matter when the process crashes, **the event is never lost**.

## 2.3 Position in System Architecture

The Data Bus occupies Layer 5 (Platform Services) in the Aegis OS component architecture. It serves as the communication backbone connecting all other layers.



## 2.4 Core Responsibilities

### Message Routing and Delivery

- **Subject-based routing:** Route messages using hierarchical patterns (e.g., aegis.tasks.routed)
- **Wildcard subscriptions:** Support pattern matching with * (single token) and > (multi- token) wildcards
- **At-least-once delivery:** Guarantee message delivery with acknowledgment-based confirmation
- **Exactly-once semantics:** Provide idempotent message processing for critical operations

### Message Persistence and Streaming

- **Stream storage:** Maintain append-only message logs with configurable retention
- **Message replay:** Allow consumers to replay messages from arbitrary points in time
- **Durable subscriptions:** Preserve consumer state across reconnections
- **Retention management:** Automatic expiration based on age, size, or message count

### Security and Isolation

- **Namespace isolation:** Separate message spaces per tenant/agent
- **Authentication:** Verify identity via mTLS or NKey/JWT tokens
- **Authorization:** Enforce fine-grained access control on subject patterns
- **Encryption:** End-to-end encryption for sensitive payloads

## 2.5 Design Principles

> **Principle 1: Zero-Trust Communication**
>
> All messages must be authenticated and authorized regardless of source. No component is implicitly trusted.

> **Principle 2: Agent Isolation**
>
> Messages from different agents are strictly isolated. Cross-agent access requires explicit authorization.

**Principle 3: Durable by Default**

All messages are persisted by default. No message is lost due to consumer unavailability.

**Principle 4: Observable Operations**

All message flows are observable through metrics, tracing, and logging.

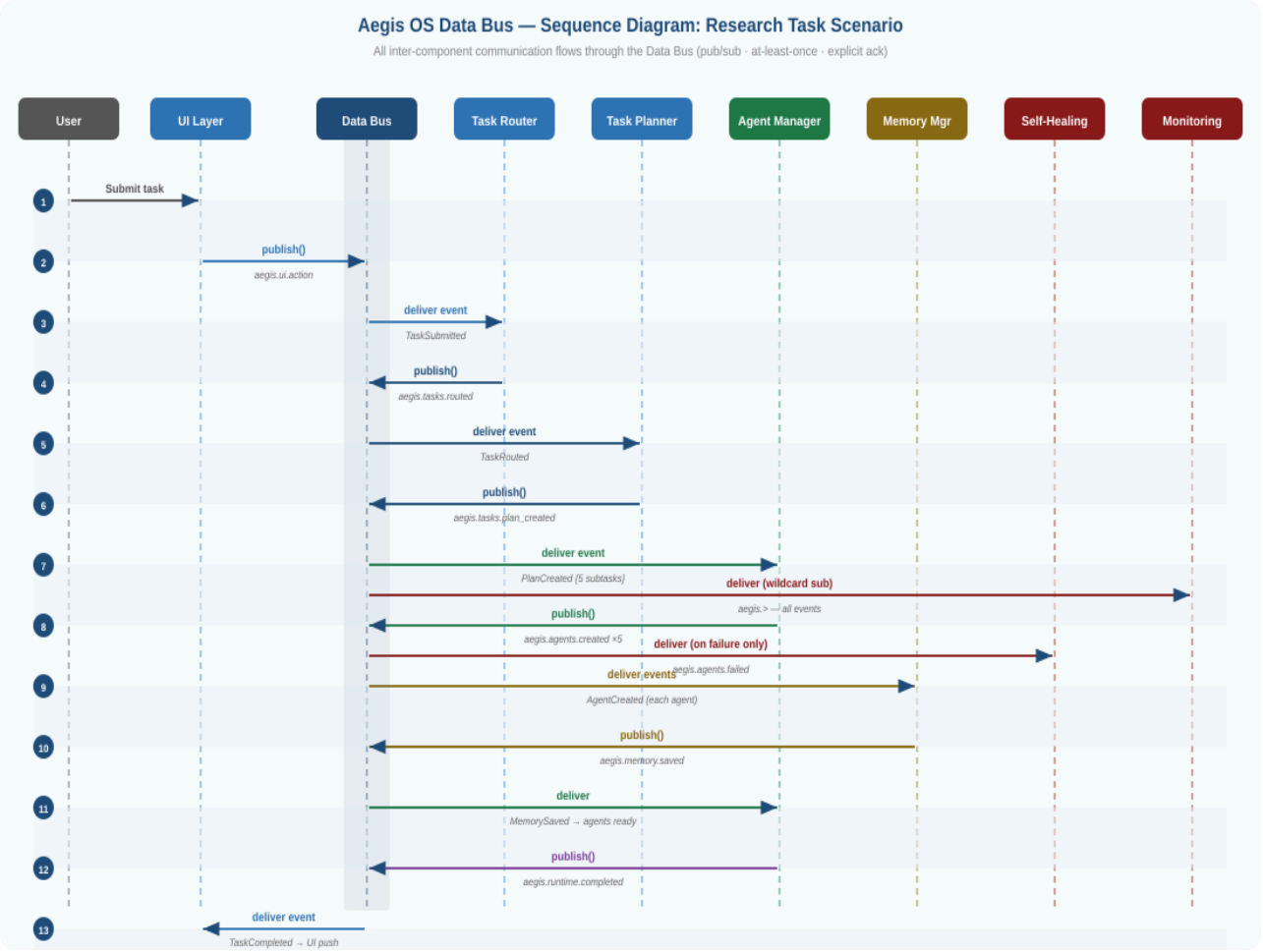
**Principle 5: Minimal Overhead**

Message transport overhead must be minimal. Target latency under 5ms for intra-node messaging.

# 3. Sequence Diagram

## 3.1 Research Task — End-to-End Event Flow

The sequence diagram below shows how the Data Bus mediates ALL communication during a complex multi-agent research task. No component calls another directly  every interaction is a publish or subscribe on the bus. The Data Bus column (dark blue) is highlighted to emphasise that it is the sole conduit.



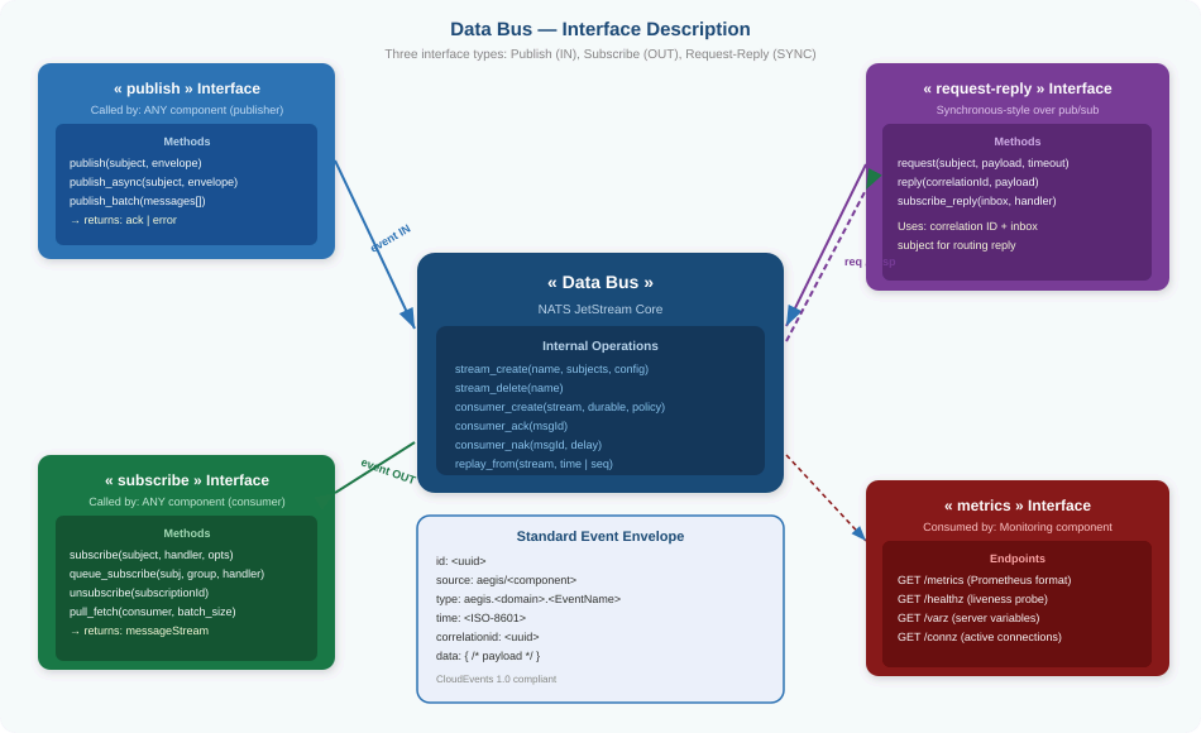
## 3. Sequence Diagram

| Step | From | To | Event / Action | Stream |
|---|---|---|---|---|
| 1 | User | UI Layer | User submits research task via dashboard | — |
| 2 | UI Layer | Data Bus | publish(aegis.ui.action, TaskSubmitted) | — |
| 3 | Data Bus | Task Router | deliver TaskSubmitted to subscriber | — |
| 4 | Task Router | Data Bus | publish(aegis.tasks.routed, {taskId, complexity}) | AEGIS_TASKS |
| 5 | Data Bus | Task Planner | deliver TaskRouted | AEGIS_TASKS |
| 6 | Task Planner | Data Bus | publish(aegis.tasks.plan_created, {5 subtasks}) | AEGIS_TASKS |
| 7 | Data Bus | Agent Manager | deliver PlanCreated | AEGIS_TASKS |
| 8 | Agent Manager | Data Bus | publish(aegis.agents.created) × 5 agents | AEGIS_AGENTS |
| 9 | Data Bus | Memory Manager | deliver AgentCreated (each agent) | AEGIS_AGENTS |
| 10 | Memory Manager | Data Bus | publish(aegis.memory.saved) per agent | AEGIS_MEMORY |
| 11 | Data Bus | Agent Manager | deliver MemorySaved → agents proceed | AEGIS_MEMORY |
| 12 | Agent Manager | Data Bus | publish(aegis.runtime.completed, {results}) | AEGIS_RUNTIME |
| 13 | Data Bus | UI Layer | deliver TaskCompleted | AEGIS_TASKS |
| 14 | UI Layer | User | Real-time dashboard push (WebSocket) | — |

# 4. Interface Description

## 4.1 Interface Overview

The Data Bus exposes four distinct interface types to the rest of the Aegis OS. Every component interacts exclusively through one or more of these interfaces — there is no shared memory or direct function calling between components.



## 4.2 Interface Specifications

### Interface 1: Publish (Write — Event IN)

Any Aegis OS component can call this interface to emit an event onto the bus. The Data Bus persists the message to the appropriate JetStream stream and routes it to all matching subscribers.

| Method | Parameters | Returns | Used By |
|---|---|---|---|
| `publish(subject, envelope)` | subject: string, envelope: CloudEvent JSON | ack: {seq, stream, duplicate} | All 18 components |
| `publish_async(subject, envelope)` | subject: string, envelope: CloudEvent JSON | Promise<ack> | High-throughput publishers |
| `publish_batch(messages[])` | messages: [{subject, envelope}] | ack[] \| partial_error | Data Ingestion Pipeline |

### Interface 2: Subscribe (Read — Event OUT)

Components subscribe to receive events. The Data Bus delivers matching messages asynchronously. Explicit acknowledgment is required for all JetStream consumers; unacknowledged messages are redelivered after AckWait.

| Method | Parameters | Returns | Used By |
|---|---|---|---|
| `subscribe(subject, handler, opts)` | subject: string (wildcards ok), handler: fn, opts: {durable, ackPolicy} | subscriptionId | Most components |
| `queue_subscribe(subject, group, handler)` | subject: string, group: string, handler: fn | subscriptionId | Scaled Agent Manager, Runtime |
| `pull_fetch(consumer, batchSize)` | consumer: durableName, batchSize: int | messages[] | Data Ingestion Pipeline |
| `unsubscribe(subscriptionId)` | subscriptionId: string | void | Any component on shutdown |

### Interface 3: Request-Reply (Synchronous over Pub/Sub)

Implements a synchronous call pattern on top of the async bus using correlation IDs and ephemeral inbox subjects. The caller blocks (with timeout) until a reply arrives.

| Method | Parameters | Returns | Used By |
|---|---|---|---|
| `request(subject, payload, timeout)` | subject: string, payload: JSON, timeout: duration | reply: JSON \| TimeoutError | Task Router → Personalization, Agent Mgr → Config Mgr |
| `reply(correlationId, payload)` | correlationId: string, payload: JSON | void | Config Mgr, Permission Mgr (responders) |

### Interface 4: Metrics (Observability OUT)

The Data Bus exposes operational metrics via HTTP endpoints. The Monitoring & Observability component polls these endpoints.

| Endpoint | Protocol | Format | Key Metrics |
|---|---|---|---|
| `GET /metrics` | HTTP | Prometheus text | msgs_published_total, msgs_delivered_total, stream_bytes, consumer_lag |
| `GET /healthz` | HTTP | JSON {ok: bool} | Liveness probe — used by Self-Healing Controller |
| `GET /varz` | HTTP | JSON | Server version, config, uptime, memory usage |
| `GET /connz` | HTTP | JSON | Active connections per component NKey |
| `GET /jsz` | HTTP | JSON | JetStream stream stats: messages, bytes, consumer count |

## 4.3 Standard Event Envelope (CloudEvents)

All messages published to the Data Bus must conform to the CloudEvents 1.0 envelope schema. The Data Bus enforces schema validation on core Aegis subjects and rejects malformed envelopes at ingress.

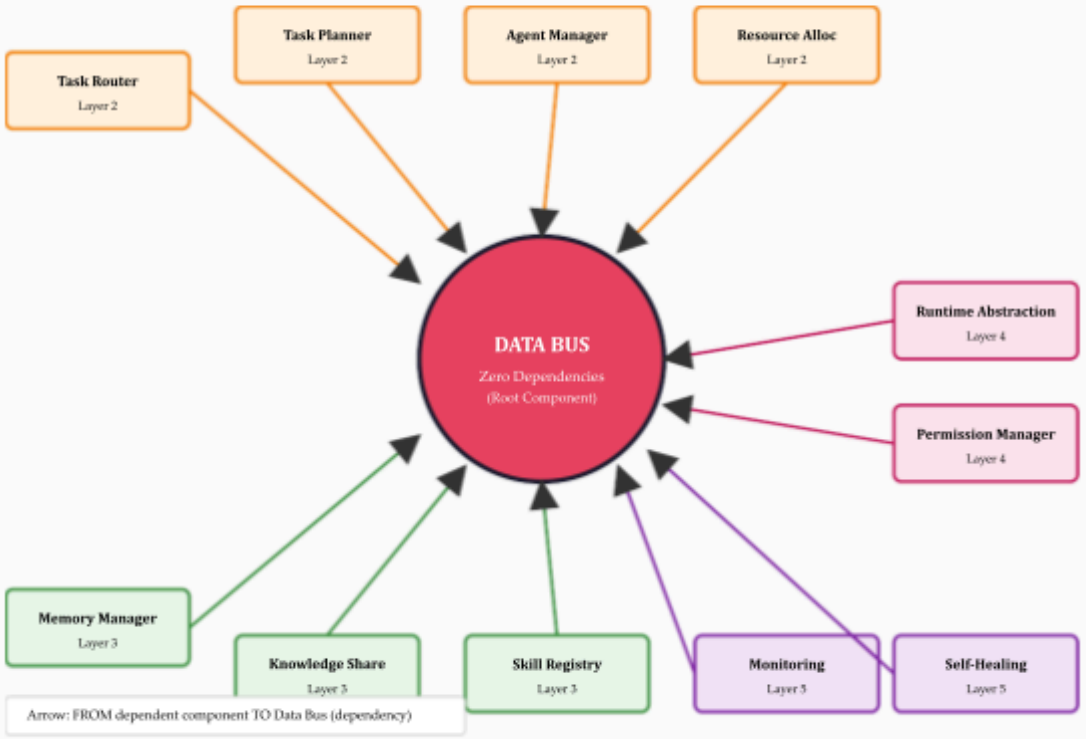
```
envelope = {
  specversion: "1.0",    // CloudEvents version — always "1.0"
  id: uuid(),      // Unique event ID — used for deduplication source:
  "aegis/task-router", // Component identity — matches NKey auth type:
  "aegis.tasks.routed",  // Subject mirror — for schema registry lookup time:
  ISO8601(),       // Event timestamp
  correlationid: uuid(), // Links related events (task lifecycle)
  datacontenttype: "application/json",
  data: {   // Event-specific payload taskId: "...",
    userId: "...",
    // ...
  }
}
```

# 5. Dependency Description

## 5.1 Component Dependency Overview

The Data Bus is a **zero-dependency** component in the Aegis OS. It must start first before any other component can initialise. Every other component depends on the Data Bus either directly (to publish/subscribe) or transitively.

**Figure 5-1: Component Dependency Diagram**



## 5.2 Boot Sequencing from Dependencies

The dependency graph directly defines the mandatory boot order for Aegis OS.

| Boot Order | Component(s) | Waits For | Rationale |
|---|---|---|---|
| 1 | Data Bus (NATS JetStream) | Nothing | Zero-dependency root. Must be ready before anything else starts. |
| 2 | Config Manager, OpenBao | Data Bus | All components need configuration at startup; secrets vault needed by Permission Manager. |
| 3 | Monitoring, Self-Healing | Data Bus, Config Manager | Monitoring subscribes to aegis.> immediately — must be up before events flow. |
| 4 | Runtime Abstraction, Permission Manager, Ext. Connectors | Data Bus, Config, OpenBao | Layer 4 runtime services must be ready before agents can be spawned. |
| 5 | Task Router, Task Planner, Agent Manager, Resource Allocator | Data Bus + Layer 4 | Core orchestration layer — can now route and plan tasks safely. |
| 6 | Memory Manager, Knowledge Sharing, Skill Registry, Data Ingestion | Data Bus + Layer 2 | Agent capability layer — depends on orchestration being functional. |
| 7 | UI Layer, Personalization Engine | All layers below | User-facing layer — starts last, when full system is operational. |

## 5.3 Failure Recovery

Because every component depends on the Data Bus, its failure is a Severity-1 system-wide incident. The Self-Healing Controller monitors the Data Bus via heartbeat on `aegis.health.databus` and triggers the following recovery sequence:

1. Self-Healing detects Data Bus heartbeat timeout → classifies as P1 failure
2. Raft consensus in NATS cluster elects a new leader node within 5 seconds
3. All components reconnect automatically (NATS client reconnect with exponential backoff)
4. JetStream stream state is recovered from Raft log — no message loss for acked messages
5. Components replay unprocessed messages from their durable consumer sequence
6. Self-Healing publishes RecoveryCompleted to `aegis.health.recovery_completed`
7. Monitoring records incident duration and updates SLA metric

# 6. Requirements Specification

## 6.1 Functional Requirements

| ID | Requirement | Priority | Acceptance Criterion |
|---|---|---|---|
| FR-DB-001 | Publish-Subscribe: Any component publishes to a subject; all matching subscribers receive the event asynchronously. | MUST | 100 msgs published; all subscribers receive within 5ms P99 |
| FR-DB-002 | Request-Reply: Synchronous-style call over pub/sub using correlation IDs and inbox subjects. | MUST | request() returns reply within 5s timeout; TimeoutError if no reply |
| FR-DB-003 | Durable Persistence: All events persisted to file- backed JetStream storage. Survive node restarts. | MUST | Kill node; restart; all unacknowledged msgs redelivered. Zero loss. |
| FR-DB-004 | Queue Groups: Messages distributed to exactly one member of a named group for horizontal scaling. | MUST | 3 Agent Manager instances in group; each message delivered to exactly 1 |
| FR-DB-005 | Wildcard Routing: Hierarchical subjects with * (single token) and > (multi-level) wildcards. | MUST | aegis.agents.> receives created, failed, terminated events |
| FR-DB-006 | Priority Queues: High-priority events processed before standard-priority under load. | SHOULD | HealthCheckFailed (high) processed before ResourceMetrics (standard) |
| FR-DB-007 | Cluster Distribution: Events published on Node A received by subscribers on Node B/C. | MUST | 3-node cluster: Node failure; leader election < 5s; no message loss |
| FR-DB-008 | Event Replay: Consumers subscribe from historical sequence or timestamp. | MUST | Self-Healing replays last 100 agent events from T-10min on restart |
| FR-DB-009 | Ack & Retry: unacknowledged messages redelivered after AckWait. Dead-letter after MaxDeliver. | MUST | Crash consumer mid-process; msg redelivered in 30s; DLQ after 5 attempts |
| FR-DB-010 | Schema Validation: Reject malformed CloudEvent envelopes on core subjects. | SHOULD | Missing required id field → publish() returns ValidationError |
| FR-DB-011 | Outbox Pattern: libSQL atomic write + relay ensures zero-loss on publish crash. | MUST | Kill process between DB write and NATS publish; relay republishes on restart |

## 6.2 Non-Functional Requirements

| ID | Category | Requirement | Target |
|---|---|---|---|
| NFR-DB - 001 | Throughput | Sustained message rate on single node | ≥ 50,000 msgs/sec |
| NFR-DB - 002 | Latency | End-to-end publish-to-receive, in- cluster | < 5ms P99 under normal load |
| NFR-DB - 003 | Availability | Uptime in cluster mode (3+ nodes) | 99.99% |
| NFR-DB - 004 | Durability | Message loss after ack | Zero (0 msgs) |
| NFR-DB - 005 | Startup Time | Time from process start to ready state | < 3 seconds |
| NFR-DB - 006 | Security | All connections encrypted | TLS 1.3 mandatory; plaintext rejected |
| NFR-DB - 007 | Auth | Per-component authentication | Unique NKey (Ed25519) per component |
| NFR-DB - 008 | Observability | Metrics export format | Prometheus /metrics endpoint |
| NFR-DB - 009 | Retention | Stream storage bounds | 7 days OR 10 GB, whichever first |
| NFR-DB - 010 | Scalability | Throughput scaling with cluster nodes | Near-linear horizontal scaling |

## 6.1 Security Requirements

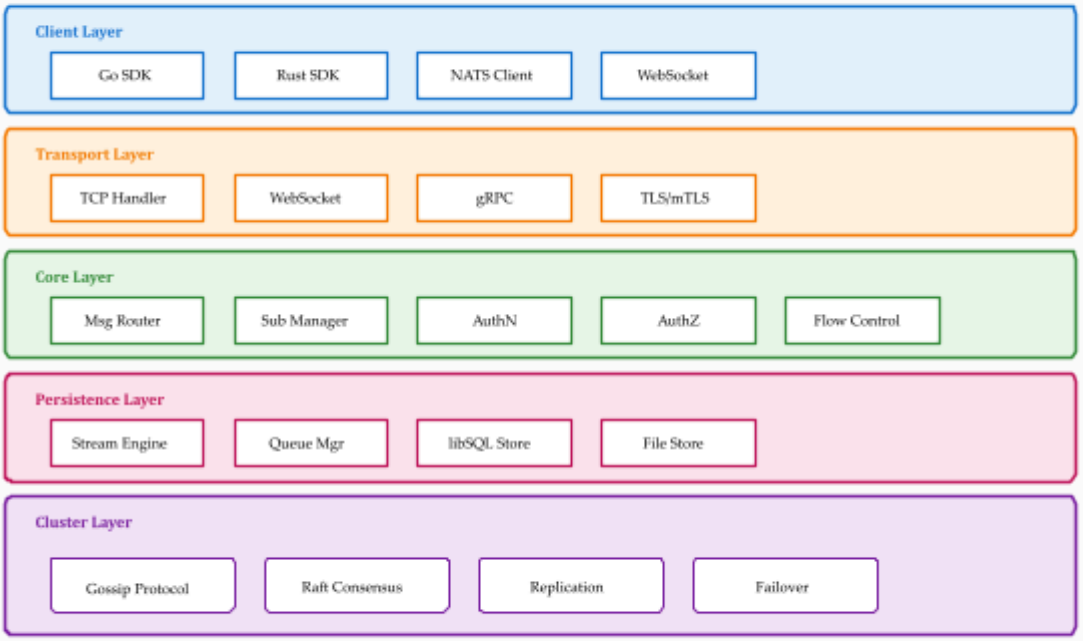
- **SR-DB-001:** TLS 1.3 mandatory for all NATS connections. Plaintext rejected with auth error.
- **SR-DB-002:** Each component authenticates using a unique NKey (Ed25519). Shared credentials prohibited.
- **SR-DB-003:** Subject-level ACLs restrict each component to only its authorised subjects (publish + subscribe separately controlled).
- **SR-DB-004:** NKeys stored and rotated by OpenBao secrets vault. Never written to disk in plaintext.
- **SR-DB-005:** Message payloads not logged in production. Only metadata (subject, size, timestamp, correlationId) logged.
- **SR-DB-006:** Dead-letter queue (aegis.dlq.>) accessible only to administrative consumers.

Not subscribable by regular components.

# 7. Detailed Design

## 7.1 Architecture Overview

**Figure 7-1: Data Bus Layered Architecture**



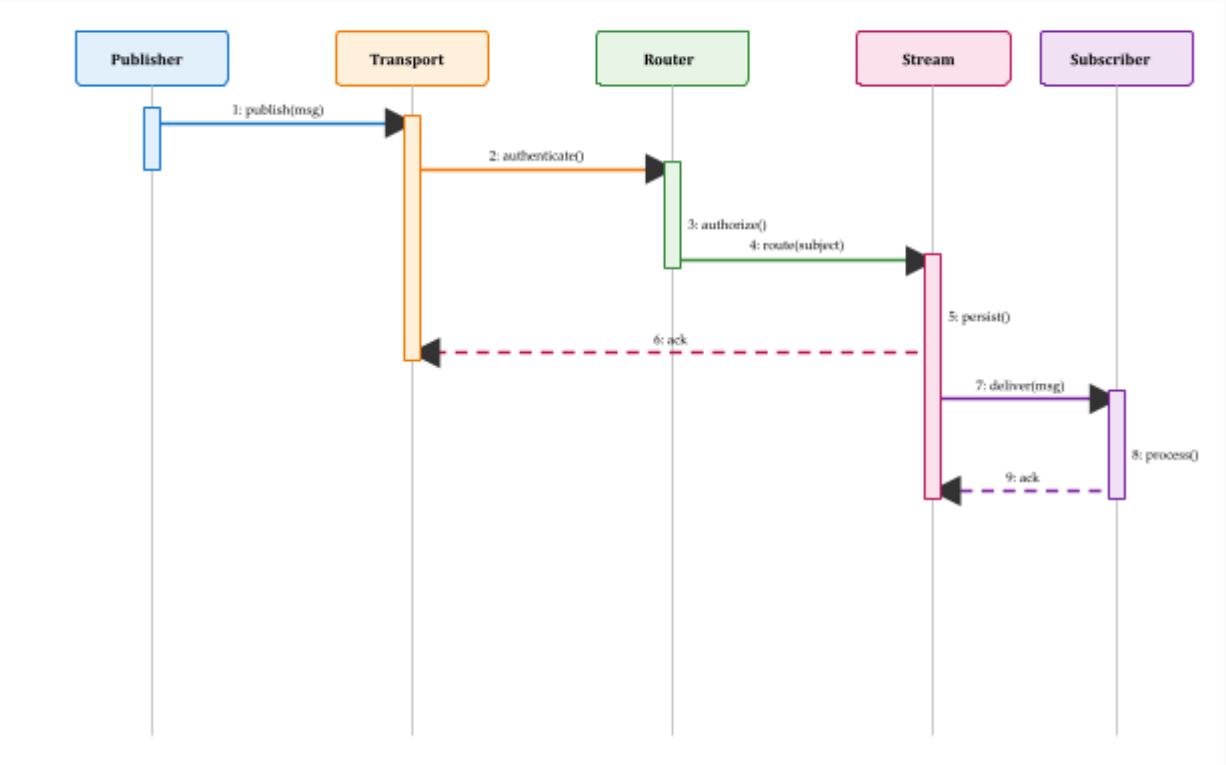

## 7.2 Component Interactions

**Figure 7-2: Component Interaction Diagram**

## 7.3 Message Flow and Sequences

### 7.3.1 Message Publishing Sequence

**Figure 7-3: Message Publishing Sequence Diagram**



## 7.4 Data Structures

### 7.4.1 Message Structure

```
{
  "header": {
    "message_id": "uuid-v4-string",
    "correlation_id": "uuid-v4-string",
    "timestamp": "2026-02-18T10:30:00Z",
    "subject": "aegis.tasks.routed",
    "content_type": "application/json",
    "encoding": "none|gzip|snappy",
    "priority": 5,
    "ttl_seconds": 3600
  },
  "payload": {
    // Application-specific data (max 8MB)
  },
  "metadata": {
    "publisher_id": "task-router",
    "publisher_ip": "10.0.0.5",
    "trace_id": "trace-abc-123"
  }
}
```

### 7.4.2 Stream Configuration

```
StreamConfig {
    Name:             "AEGIS_TASKS"
    Subjects:         ["aegis.tasks.>"]
    Retention:        "limits"        // limits | interest | workqueue
    MaxMsgs:          1000000         // Maximum messages
    MaxBytes:         1073741824      // 1GB
    MaxAge:           168h            // 7 days
    Storage:          "file"          // file | memory
    Replicas:         3               // Replication factor
    Discard:          "old"           // old | new
    DuplicateWindow:  2m              // Deduplication window
}
```

# 8. OpenClaw Variant Analysis

## 8.1 Comparative Analysis

OpenClaw is an open-source autonomous AI agent framework that has gained significant traction. The framework enables developers to deploy self-hosted AI agents that autonomously perform tasks through various messaging platforms.

## 8.2 Security Lessons (CVE-2026-25253)

**Vulnerability Summary: CVE-2026-25253**

The `/api/export-auth` endpoint in OpenClaw allowed **unauthenticated extraction of all stored API tokens** (Claude, OpenAI, Google AI keys) due to missing authentication checks.

**Impact:** Critical - Complete compromise of all connected AI services

**Aegis OS Mitigations:**

All administrative endpoints require mTLS + JWT authentication Credential export requires multi-factor confirmation
Audit logging for all credential access attempts
Rate limiting on sensitive operations
Principle of least privilege for all Data Bus operations

| OpenClaw Pattern | Aegis OS Adaptation | Benefit |
|---|---|---|
| Single-process gateway | Microservice-based Data Bus | Better isolation, scalability |
| File-based memory | libSQL-backed streams | ACID guarantees, replication |
| Heartbeat scheduling | DurableTask-Go integration | Fault-tolerant execution |
| Multi-channel support | Subject-based routing | Flexible, extensible patterns |
| Skill registry | Agent Skill Registry | Standardized capabilities |

# 9. Proof of Concept

## 9.1 PoC Components

| Component | Technology | Purpose |
|---|---|---|
| Message Broker | NATS Server 2.10+ | Core message routing |
| Persistence Layer | JetStream | Stream storage and replay |
| Storage Backend | Local filesystem | Message file storage |
| API Gateway | Go + Gin | HTTP/WebSocket interface |
| Client SDK | Go (nats.go) | Application integration |
| Monitoring | Prometheus + Grafana | Metrics and dashboards |

## 9.2 PoC Pseudocode

### Stream Setup (runs once at Aegis boot)

```
// aegis-databus-setup
FOR each domain IN [TASKS, AGENTS, RUNTIME, VAULT, MEMORY, MONITORING]:
  data_bus.create_stream(
    name     = "AEGIS_" + domain,
    subjects = ["aegis." + domain.lower() + ".>"],
    storage  = FILE,
    max_age  = 7_DAYS,
    max_bytes = 10_GB,
    replicas = 1  // → 3 in cluster mode
  )
```

### Publisher with Zero-Loss Outbox

```
// mock-task-router — publish with outbox guarantee
FUNCTION publishTaskRouted(taskId, userId):
  envelope = build_cloud_event(
    source  = "aegis/task-router",
    type    = "aegis.tasks.routed",
    data    = { taskId, userId, complexity: "high" }
  )

  // Step 1: atomic write to local outbox (survives crash)
  libsql.BEGIN()
  libsql.insert(outbox, subject="aegis.tasks.routed", payload=envelope)
  libsql.COMMIT()

  // Step 2: publish to NATS (Outbox Relay retries if this fails)
  ack = data_bus.publish("aegis.tasks.routed", envelope)
  libsql.update(outbox, status='sent', seq=ack.seq)
```

### Consumer with Queue Group + Explicit Ack

```
// mock-agent-manager — queue group subscriber
data_bus.queue_subscribe(
  subject = "aegis.tasks.routed",
  group   = "agent-managers",  // competing consumers
  handler = FUNCTION(msg):
    result = spawn_agents(msg.data.taskId, msg.data.subtasks)
    IF result.ok:
      data_bus.publish("aegis.agents.created", build_event(result))
      msg.ack()                  // remove from JetStream pending
    ELSE:
      msg.nak(delay=5s)           // redelivered after 5s backoff
  opts = { durable="agent-manager", ack_wait=30s, max_deliver=5 }
)
```

## 9.3 PoC Test Cases

| ID | Test Case | Input | Expected Output | Pass Criteria |
|---|---|---|---|---|
| TC-001 | Single Publish-Subscribe | Publish 1 msg to aegis.test | Subscriber receives msg | Delivery latency < 5ms |
| TC-002 | Queue Group Distribution | Publish 100 msgs, 3 subscribers in group | Each subscriber receives ~33 msgs | No msg delivered to multiple subscribers |
| TC-003 | Durable Consumer Recovery | Kill subscriber at msg 50, restart | Subscriber resumes from msg 51 | Zero messages lost or duplicated |
| TC-004 | Stream Replay | Create consumer with deliver_all, 1000 msgs in stream | All 1000 msgs replayed | Replay completes in < 2s |
| TC-005 | Cluster Failover | Kill leader node during 1000 msg burst | New leader elected, msgs continue | < 5s downtime, zero msg loss |
| TC-006 | Outbox Zero-Loss | Kill process between DB write and NATS publish | Msg published on restart | 100% delivery guarantee |

# 10. High-Level User Task Scenario

## 10.1 Task Definition

**User Task: Multi-Agent Research Workflow**

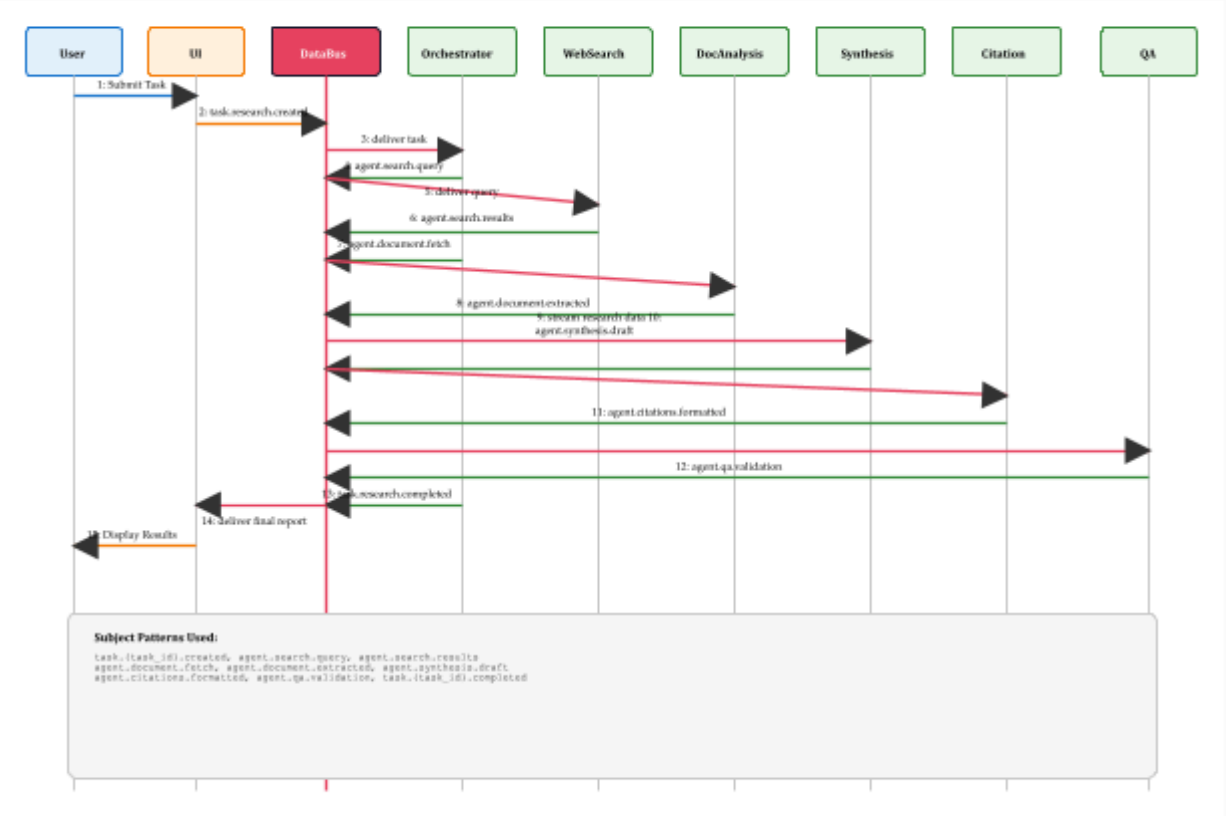
A user requests Aegis OS to conduct comprehensive research on a topic (e.g., "emerging AI security threats in 2026"). The system orchestrates multiple specialized agents that collaborate through the Data Bus to gather, analyze, and synthesize information.

### Task Components

| Agent | Role | Data Bus Interactions |
|---|---|---|
| Orchestrator Agent | Task coordination | Publishes task assignments, subscribes to agent status |
| Web Search Agent | Internet research | Publishes search results, consumes search queries |
| Document Analysis Agent | PDF/document processing | Publishes extracted content, consumes document URLs |
| Knowledge Synthesis Agent | Information correlation | Consumes all research outputs, publishes final report |
| Citation Manager Agent | Source tracking | Consumes all sources, publishes formatted citations |
| Quality Assurance Agent | Fact-checking | Consumes draft report, publishes validation results |

## 10.2 Event Flow

**Figure 10-1: Multi-Agent Research Workflow Sequence**



# 11. Roadmap & Success Criteria

## 11.1 Implementation Phases

| Phase | Duration | Deliverables | Key Milestone |
|---|---|---|---|
| **Phase 0: PoC** | Week 1 | Single-node NATS + JetStream; Go SDK; 6 stream definitions; basic pub/sub | TC-001, TC-002 passing |
| **Phase 1: Core** | Week 2 | Queue groups; durable consumers; request- reply; outbox relay | TC-003, TC-004, TC-006 passing |
| **Phase 2: Cluster** | Week 3 | 3-node Raft cluster; automatic failover; replication tuning | TC-005 passing; 99.99% uptime |
| **Phase 3: Security** | Week 4 | TLS 1.3; NKey auth; subject ACLs; audit logging | Security audit passed |
| **Phase 4: Integration** | Week 5 | All 18 components connected; end-to-end task flow; monitoring | Research task completes in < 60s |
| **Phase 5: Production** | Week 6 | Performance tuning; chaos testing; documentation; runbooks | 50K msgs/sec sustained; 5ms P99 latency |

## 11.2 Success Criteria by Phase

| Phase | Functional | Performance | Security |
|---|---|---|---|
| PoC | Pub/sub works; queue groups distribute | < 10ms latency (single node) | None (plaintext OK) |
| Core | Durables survive restart; replay works | < 5ms latency | Basic auth token |
| Cluster | Failover < 5s; zero msg loss | Linear scale with nodes | TLS 1.3 |
| Security | ACLs enforced; audit logs complete | Auth adds < 1ms overhead | Pen test passed |
| Integration | All 18 components connected | End-to-end < 60s | mTLS everywhere |
| Production | Zero incidents in chaos test | 50K msgs/sec; 5ms P99 | Compliance audit passed |

# 12. Risks and Mitigation

| Risk | Impact | Likelihood | Mitigation Strategy |
|---|---|---|---|
| Message loss during node failure | Critical | Low | 3x replication, synchronous acks, RAID storage |
| Network partition causing split- brain | High | Medium | Raft consensus, majority quorum, automatic healing |
| Unauthorized message access | Critical | Low | mTLS, JWT auth, subject-level ACLs, audit logging |
| Performance degradation under load | Medium | Medium | Horizontal scaling, backpressure, flow control |
| Message ordering violations | High | Low | Single-partition writers, sequence numbers |
| Storage exhaustion | High | Medium | Retention policies, monitoring alerts, auto- expansion |
| Client library incompatibility | Medium | Low | NATS protocol compatibility, version pinning |
| Operator error (config, deletion) | High | Medium | Config validation, confirmation prompts, backups |
| Dependency on NATS project health | Medium | Low | CNCF project, multiple vendors, fork capability |
| Complexity overwhelming operators | Medium | Medium | Documentation, automation, observability |

# 13. Appendix

## 13.1 Glossary

| Term | Definition |
|---|---|
| Ack | Acknowledgment from consumer confirming successful message processing |
| CloudEvents | CNCF event envelope specification: specversion, id, source, type, time, data |
| Consumer | Stateful JetStream entity representing a named subscription into a Stream |
| Data Bus | Central message transport infrastructure enabling component communication |
| DLQ | Dead-Letter Queue — messages exceeding MaxDeliver are routed here |
| Durable | Consumer whose state persists across reconnections |
| JetStream | NATS persistence/streaming layer — adds durable Streams and Consumers |
| NKey | Ed25519 keypair used for NATS component authentication |
| Outbox | libSQL table storing events atomically with business data |
| Queue Group | Named subscriber group where each message is delivered to exactly one member |
| Replay | Replaying historical stream messages from a sequence offset or timestamp |
| Stream | Ordered, persistent log of messages on matching NATS subjects |
| Subject | Hierarchical routing key (e.g., aegis.tasks.routed) — supports * and > wildcards |

## 13.2 References


1. Aegis OS Official Specification, Aegis Team, February 2026.
2. NATS Documentation, *https://docs.nats.io/*, Synadia Communications.
3. NATS JetStream Documentation, *https://docs.nats.io/nats-concepts/jetstream*.
4. OpenClaw GitHub Repository, *https://github.com/openclaw*, MIT License.
5. CVE-2026-25253: Authentication Bypass in OpenClaw Agent Gateways, Hunt.io Security Advisory, February 2026.
6. Badre, D. (2020). *On Task: How Our Brain Gets Things Done*. Princeton University Press.
7. CloudEvents Specification 1.0, CNCF, *https://cloudevents.io/*.
8. Firecracker MicroVM Documentation, *https://firecracker-microvm.github.io/*, AWS.

9. DurableTask Framework, *https://github.com/microsoft/durabletask-go*, Microsoft.
10. libSQL Documentation, *https://github.com/tursodatabase/libsql*, Turso.
11. OpenBao Documentation, *https://openbao.org/docs/*, OpenBao Project.
12. gRPC Documentation, *https://grpc.io/docs/*, CNCF.
13. Prometheus Monitoring, *https://prometheus.io/docs/*, CNCF.
14. OpenTelemetry Documentation, *https://opentelemetry.io/docs/*, CNCF.